

Laboratory 5 Report

ETL Process for the Flight Delays Data Warehouse

Piotr Krupiński, Jeremi Szczotka, Szymon Mańka

June 2026

Contents

1	Analysis of Data Sources	2
1.1	Dataset 3 — Primary Flight Data (2015)	2
1.2	Dataset 1 — Supplementary Flight Data (2019–2023)	3
1.3	Dataset 2 — Excluded	3
2	ETL Approach	4
2.1	Chosen Approach	4
2.2	Rationale	4
2.3	Technology Stack	4
2.4	Pipeline Overview	5
3	Data Staging Area	5
4	Transformations	5
4.1	Dim_Date — Programmatic Generation	5
4.2	Dim_Airline	6
4.3	Dim_Airport	6
4.4	Dim_Plane	7
4.5	Dim_Cancellation_Reason	7
4.6	Column Normalisation (Unified Schema)	7
4.7	Fact Table Transformations (Per-Chunk)	8
5	Loading Phase	9
5.1	Schema Creation	9
5.2	Dimension Loading	9
5.3	Lookup Dictionary Build	10
5.4	Fact Table Loading — COPY FROM STDIN	10
5.5	Split and Join Operations Required by the Warehouse Schema	10
5.6	Final Row Counts	11

1 Analysis of Data Sources

Three datasets were considered for this project. Before beginning the ETL design, each was analysed for quality issues, structural incompatibilities, and missing data.

1.1 Dataset 3 — Primary Flight Data (2015)

Dataset 3 is the primary source and consists of three CSV files located in `data/dataset3/`:

- `flights.csv` — approximately 5.8 million rows of US domestic flight records for the year 2015, with 31 columns covering scheduling, delays, cancellations, and operational metrics.
- `airlines.csv` — 14 rows mapping IATA carrier codes to full airline names.
- `airports.csv` — 322 rows with IATA airport codes, names, cities, states, and geographic coordinates.

The following quality issues were identified and resolved:

Issue	Column(s)	Rows affected	Resolution
Missing departure and arrival delays	DEPARTURE_DELAY, ARRIVAL_DELAY	0pt~86 153	Expected for cancelled flights; stored as NULL
Missing tail number	TAIL_NUMBER	0pt~14 721	Mapped to UNKNOWN sentinel row in Dim_Plane
All delay breakdown columns empty	AIR_SYSTEM_DELAY through WEATHER_DELAY	0pt~4 755 640	Stored as 0 — absence of delay is not a defect
Cancellation code absent on non-cancelled flights	CANCELLATION_REASON	0pt~5 729 195	Mapped to sentinel code N (Not Applicable) in Dim_Cancellation_Reason
Times stored as HHMM integer strings	SCHEDULED_DEPARTURE, DEPARTURE_TIME, ARRIVAL_TIME, SCHEDULED_ARRIVAL	All rows	Converted to HH:MM:00 format; edge case 2400 mapped to 00:00:00
Date split across three separate columns	YEAR, MONTH, DAY	All rows	Reconstructed into a single DATE value and integer <code>date_key</code>
No aircraft metadata	TAIL_NUMBER only	All rows	Dim_Plane stores tail numbers only; <code>model</code> , <code>manufacturer</code> , and <code>issue_date</code> remain NULL

A specific technical issue was encountered with HHMM time columns: because values such as 0005 or 0930 could be parsed by pandas as integers, leading zeroes would be silently

dropped. This was prevented by reading all time columns with explicit `dtype=str` in the extraction layer.

1.2 Dataset 1 — Supplementary Flight Data (2019–2023)

Dataset 1 (`data/dataset1/flights_sample_3m.csv`) contains approximately 3 million rows covering the years 2019 through 2023. Although it records the same domain (US domestic flights), its schema differs from Dataset 3, requiring normalisation before it can be fed into the same transformation pipeline.

Issue	Column(s)	Rows affected	Resolution
Different column naming conventions	All columns	All rows	Renamed to unified internal names in <code>normalize_dataset1_chunk()</code>
No <code>TAIL_NUMBER</code> column	—	All rows	<code>plane_key</code> set to the <code>UNKNOWN</code> sentinel for all Dataset 1 rows
Airlines not in <code>airlines.csv</code>	<code>AIRLINE_CODE</code>	Varies	Pre-scan of Dataset 1 inserts any missing carriers into <code>Dim_Airline</code> before fact loading
Flight date as <code>YYYY-MM-DD</code> string	<code>FL_DATE</code>	All rows	Parsed directly; no HHMM conversion required
Delay breakdown columns differently named	e.g. <code>DELAY_DUE_CARRIER</code> vs <code>AIRLINE_DELAY</code>	All rows	Unified by column rename during normalisation

1.3 Dataset 2 — Excluded

Dataset 2 (`data/dataset2/airline.csv.shuffle`, 11 GB) was evaluated and excluded from the ETL for the following reasons:

1. **File size.** 11 GB exceeds practical processing limits for a local academic environment without a distributed computing framework.
2. **Encoding.** The file uses latin-1 (non-UTF-8) encoding, which creates normalisation complexity and risks mojibake in string columns.
3. **Missing reference data.** No airport or airline lookup tables accompany the file, making it impossible to enrich records with the geographic or carrier metadata required by the dimensional model.
4. **No data dictionary.** Some column semantics are ambiguous without formal documentation.
5. **Sufficient coverage from other sources.** Datasets 1 and 3 together cover approximately 8.8 million flight records spanning 2015 and 2019–2023, which provides

adequate volume and temporal range for all planned analyses.

The only artefact retained from Dataset 2 is `data/dataset2/carriers.csv` (1 492 airline codes), which is preserved as a potential future supplement for extending `Dim_Airline`.

2 ETL Approach

2.1 Chosen Approach

A **self-developed Python ETL** (Approach 2 from the laboratory specification) was selected. The pipeline is implemented as four modules:

- `etl/extract.py` — reads raw data from CSV files.
- `etl/transform.py` — builds dimension DataFrames and transforms raw flight records into fact rows.
- `etl/load.py` — writes transformed data to PostgreSQL.
- `etl/etl.py` — the orchestrator that invokes the above in the correct order and manages database connections.

2.2 Rationale

- Full control over transformation logic, NULL handling, and memory-efficient chunk iteration.
- The `pandas` + `psycopg2` combination is well-suited to the CSV-to-PostgreSQL stack used in this project.
- No additional licensed software or services are required.
- The code is straightforward to inspect, debug, and extend.

An ELT approach (Approach 3) was considered but rejected: it would require loading approximately 12 GB of raw CSV data into PostgreSQL before any transformations, which is less efficient than the chunked in-memory Python approach. A dedicated ETL tool (Approach 1, e.g. SSIS) was rejected because it would introduce a heavyweight dependency for a relatively straightforward pipeline.

2.3 Technology Stack

Component	Tool	Version
Extraction	<code>pandas.read_csv</code> with chunk iteration	<code>pandas</code> $\geq$ 2.0
Transformation	Python, <code>pandas</code> in-memory DataFrames	Python 3.9+
Loading	<code>psycopg2 execute_values</code> + <code>COPY FROM STDIN</code>	<code>psycopg2-binary</code> $\geq$ 2.9
Database	PostgreSQL 14 running in a Docker container	PostgreSQL 14
Orchestration	<code>etl/etl.py</code>	—

2.4 Pipeline Overview

The `run_etl()` function in `etl.py` executes the following eight steps in order:

1. Create (or recreate) the schema from `sql/ddl.sql`.
2. Build and load `Dim_Date` (programmatically generated).
3. Extract, transform, and load `Dim_Airline`.
4. Extract, transform, and load `Dim_Airport`.
5. Scan `flights.csv` for unique tail numbers; build and load `Dim_Plane`.
6. Query all dimension surrogate keys into in-memory lookup dictionaries.
7. Stream Dataset 3 flight records in 100 000-row chunks through the transformation pipeline and bulk-load into the fact table.
8. Pre-scan Dataset 1 for missing airlines; stream flight records through the same pipeline and load into the fact table.

3 Data Staging Area

No separate staging area database was used. The raw CSV files on disk serve directly as the source layer. The ETL reads data in memory-efficient chunks of 100 000 rows without first materialising the full dataset in any intermediate database.

This decision is justified because:

- The datasets are *static snapshots*, not live feeds — there is no ongoing requirement to merge incremental loads.
- The transformation logic is simple enough to be applied entirely in-memory during chunk iteration.
- Loading 12 GB into a staging database first would double the total disk I/O for no analytical benefit.

If the pipeline were extended to support incremental loads from live sources, introducing a staging schema (e.g. raw tables mirroring the CSV structure) would become appropriate.

4 Transformations

This section describes every transformation applied prior to loading data into the data warehouse, based on the issues identified in Section 1.

4.1 Dim_Date — Programmatic Generation

`Dim_Date` is *not* derived from any source file. It is generated entirely in Python by the `build_dim_date()` function in `transform.py`. For each year in the set {2015, 2019, 2020, 2021, 2022, 2023}, every calendar day is enumerated and the following attributes are computed:

Attribute	Derivation
<code>date_key</code>	$YEAR \times 10000 + MONTH \times 100 + DAY$
<code>full_date</code>	Python <code>date</code> object
<code>year</code>	<code>date.year</code>
<code>month</code>	<code>date.month</code>
<code>month_name</code>	Static lookup (e.g. 1 $\rightarrow$ 'January')
<code>quarter</code>	$(month - 1) \div 3 + 1$
<code>day_of_month</code>	<code>date.day</code>
<code>day_of_week</code>	<code>date.isoweekday()</code> — 1 = Monday, 7 = Sunday
<code>day_name</code>	Static lookup (e.g. 3 $\rightarrow$ 'Wednesday')
<code>is_weekend</code>	<code>day_of_week</code> ≥ 6
<code>season</code>	Month-to-season map: Dec/Jan/Feb = Winter, Mar–May = Spring, Jun–Aug = Summer, Sep–Nov = Autumn

This produces 2 192 rows covering all days across the six years.

4.2 Dim_Airline

Source: `data/dataset3/airlines.csv` (14 rows), supplemented by carrier codes discovered in Dataset 1 during Step 8 of the pipeline.

Transformations applied (in `transform_dim_airline()`):

- Strip leading/trailing whitespace from `IATA_CODE` and `AIRLINE`.
- Rename columns to the internal schema names (`iata_code`, `airline_name`).
- Set `country_of_origin` = 'USA' for all rows (all carriers in the dataset are US domestic operators).
- Deduplicate by `iata_code` to prevent duplicate inserts.

Dataset 1 supplement: Before loading Dataset 1 fact rows, the `_ensure_dataset1_airlines()` function scans the `AIRLINE_CODE` column in chunks and inserts any IATA codes not yet present in `Dim_Airline`, using the `AIRLINE` full-name column as the carrier name. Airline names longer than 50 characters are truncated to fit the `VARCHAR(50)` column constraint.

4.3 Dim_Airport

Source: `data/dataset3/airports.csv` (322 rows).

Transformations applied (in `transform_dim_airport()`):

- Strip whitespace from all string columns (`IATA_CODE`, `AIRPORT`, `CITY`, `COUNTRY`).
- Truncate `airport_name` to 100 characters (the longest observed name is 78 characters, so no data is lost in practice).
- Convert `LATITUDE` and `LONGITUDE` to `DECIMAL(9,5)` using `pd.to_numeric(errors='coerce')`; any non-numeric value is coerced to `NULL`.
- Deduplicate by `iata_code`.

4.4 Dim_Plane

Source: The TAIL_NUMBER column scanned from `data/dataset3/flights.csv` in a single pass of 100 000-row chunks.

Transformations applied (in `build_dim_plane()`):

- Extract all unique, non-null, non-empty tail number strings ($\approx 4\,800$ distinct values).
- Prepend a special UNKNOWN sentinel row as the first entry. This row acts as a foreign-key target for all fact rows that lack a tail number.
- All metadata columns (`model`, `manufacturer`, `issue_date`) are set to NULL because the source dataset provides no aircraft metadata beyond the tail number itself.
- `status` is set to the string 'Unknown' for all rows.

4.5 Dim_Cancellation_Reason

This dimension is seeded *directly in the DDL* with five static values and requires no ETL step:

Code	Description
A	Carrier
B	Weather
C	National Air System
D	Security
N	Not Applicable (sentinel for non-cancelled flights)

4.6 Column Normalisation (Unified Schema)

Because Datasets 1 and 3 use different column names for the same concepts, each chunk is first passed through a normalisation function before entering the shared fact transformation logic.

`normalize_dataset3_chunk()` (in `transform.py`) renames the Dataset 3 columns:

Source column	Internal name
AIRLINE	airline_iata
ORIGIN_AIRPORT	origin_iata
DESTINATION_AIRPORT	dest_iata
DEPARTURE_DELAY	dep_delay
ARRIVAL_DELAY	arr_delay
AIR_SYSTEM_DELAY	nas_delay
AIRLINE_DELAY	carrier_delay
CANCELLATION_REASON	cancel_code
...	...

`normalize_dataset1_chunk()` applies an analogous renaming for Dataset 1 (e.g. `AIRLINE_CODE` $\rightarrow$ `airline_iata`, `DELAY_DUE_CARRIER` $\rightarrow$ `carrier_delay`) and additionally sets `tail_number` = None for all rows (Dataset 1 has no tail number column) and copies `FL_DATE` into an auxiliary `flight_date_str` column for date parsing.

4.7 Fact Table Transformations (Per-Chunk)

Each 100 000-row chunk passes through `transform_fact_chunk()` in `transform.py`. The steps are as follows:

1. Date Key Construction

For Dataset 3, the integer date key is derived directly from the source columns:

$$\text{date_key} = \text{YEAR} \times 10000 + \text{MONTH} \times 100 + \text{DAY}$$

For Dataset 1, the date string `FL_DATE` (YYYY-MM-DD) is split on the hyphen separator and the same formula is applied. Rows whose date key does not correspond to a date already loaded in `Dim_Date` are silently dropped.

2. Dimension Key Resolution

Surrogate keys for all four foreign-key dimensions are resolved using in-memory Python dictionaries (loaded in Step 6 of the pipeline). IATA codes are stripped of whitespace before lookup:

- **Airline key:** `airline_iata` $\rightarrow$ `airline_key`. Rows with unresolved airline codes are dropped.
- **Airport keys:** `origin_iata` and `dest_iata` $\rightarrow$ `origin_airport_key` and `destination_airport_key`. Rows where either airport cannot be resolved are counted as skipped and excluded from the output.
- **Plane key:** `tail_number` $\rightarrow$ `plane_key`. Any tail number not in the lookup (including explicit `None`, empty string, or the string `'nan'`) is mapped to the `UNKNOWN` sentinel key.
- **Cancellation reason key:** resolved from the single-character `cancel_code`. For non-cancelled flights (code absent or not in the lookup), the `N` sentinel key is used.

3. HHMM Time Parsing

The helper function `parse_hhmm()` converts any HHMM representation (integer, float, or string) to the format `HH:MM:00` expected by PostgreSQL's `TIME` type:

- Float decimals are stripped (e.g. `930.0` $\rightarrow$ `'0930'`).
- The string is zero-padded to four characters (`'930'` $\rightarrow$ `'0930'`).
- The edge case `'2400'` (midnight expressed as end-of-day) is mapped to `'00:00:00'`.
- Values with out-of-range hour (> 23) or minute (> 59) are returned as `None`.
- Missing or `NaN` values return `None`.

4. NULL and Zero Handling for Delay Columns

- Cancelled flights: `dep_delay` and `arr_delay` are stored as `NULL` (flight never departed or arrived).

- Non-delayed, non-cancelled flights: all five delay breakdown columns (`carrier_delay`, `weather_delay`, `nas_delay`, `security_delay`, `late_aircraft_delay`) are stored as 0.
- Delayed flights: breakdown columns retain their original values.

5. Type Casting

All delay, time, and distance columns are cast from pandas `Float32/Float64` to Python `int` (or `None` for `NaN`) via the `_to_int_or_none()` helper. This is required because PostgreSQL's `SMALLINT` columns do not accept floating-point values.

6. Source Label

A `source_dataset` string (`'dataset3'` or `'dataset1'`) is added to every row. This column is not present in either source file; it is injected by the orchestrator to allow downstream queries to filter or aggregate by data vintage.

5 Loading Phase

5.1 Schema Creation

At the start of every ETL run, `create_schema()` reads `sql/ddl.sql` and executes it against the target database. All tables are dropped in reverse foreign-key order (`CASCADE`) and recreated, ensuring a clean state. This makes the full ETL run idempotent and reproducible.

5.2 Dimension Loading

All dimension tables are loaded using `psycopg2.extras.execute_values()`, which batches multiple row tuples into a single parameterised `INSERT` statement per call, reducing network round-trips compared to per-row execution.

Every dimension insert uses the clause `ON CONFLICT DO NOTHING`, meaning:

- The ETL can be safely re-run without duplicating dimension data.
- Partial runs (e.g. interrupted after dimension loading) do not cause constraint violations on retry.

Loading order respects the absence of cross-dimension foreign-key dependencies:

1. `Dim_Date` — 2 192 rows (programmatically generated)
2. `Dim_Airline` — 14+ rows (from `airlines.csv` plus Dataset 1 supplements)
3. `Dim_Airport` — 322 rows (from `airports.csv`)
4. `Dim_Plane` — ≈ 4800 rows (scanned from `flights.csv`)
5. `Dim_Cancellation_Reason` — 5 rows (pre-seeded by DDL; no ETL step)

5.3 Lookup Dictionary Build

After all dimension tables are populated, `build_dimension_lookup_dicts()` issues five `SELECT` queries and constructs in-memory Python dictionaries mapping each business key (IATA code, tail number, cancellation code, date key) to its surrogate key. These dictionaries are used throughout fact loading for $O(1)$ key resolution without issuing a database query per row.

5.4 Fact Table Loading — COPY FROM STDIN

The fact table is loaded using PostgreSQL’s native `COPY FROM STDIN` protocol, accessed through `psycopg2`’s `cursor.copy_expert()` method. The process per chunk is:

1. A transformed list of tuples is serialised to a tab-separated in-memory `io.StringIO` buffer by the `_row_to_tsv()` helper. Python `None` values are emitted as `\N` (the PostgreSQL text-format null marker). Boolean values are written as `t` or `f`.
2. The buffer is rewound and passed to `copy_expert()` with the following `COPY` statement:

```
COPY Fact_Flight_Operations (
    date_key, airline_key, origin_airport_key,
    destination_airport_key,
    plane_key, cancel_reason_key, flight_number,
    source_dataset,
    departure_delay_min, arrival_delay_min,
    carrier_delay_min, weather_delay_min, nas_delay_min,
    security_delay_min, late_aircraft_delay_min,
    taxi_out_min, taxi_in_min, air_time_min, distance_miles,
    flight_count, cancelled_flag, diverted_flag
) FROM STDIN WITH (FORMAT TEXT, NULL '\N')
```

3. A `COMMIT` is issued after each chunk, so progress is preserved even if the process is interrupted mid-run.

The performance advantage of `COPY` over batched `INSERT` is substantial:

Loading method	Estimated time for ~5.8M rows
<code>execute_values</code> (batched <code>INSERT</code>)	2–4 hours
<code>COPY FROM STDIN</code>	10–20 minutes

5.5 Split and Join Operations Required by the Warehouse Schema

The source data schemas differ significantly from the star schema in the data warehouse. The following split/join operations were performed during loading:

Operation	Description
Role-playing airport dimension	A single <code>Dim_Airport</code> table is referenced twice from the fact table — once as <code>origin_airport_key</code> and once as <code>destination_airport_key</code> . Both keys are resolved from the same in-memory dictionary using the origin and destination IATA codes respectively.
Date reconstruction from split columns	Dataset 3 stores the date across three separate columns (<code>YEAR</code> , <code>MONTH</code> , <code>DAY</code>). These are combined into a single integer <code>date_key</code> during transformation.
Airline surrogate key join	Source files contain only the IATA carrier code string. The integer <code>airline_key</code> surrogate is resolved via the in-memory dictionary.
Plane surrogate key join	Tail number strings are mapped to integer <code>plane_key</code> values. Rows with missing or unrecognised tail numbers are mapped to the <code>UNKNOWN</code> sentinel key.
Cancellation reason join	The raw single-character cancellation code is mapped to the integer <code>cancel_reason_key</code> ; non-cancelled rows receive the <code>N</code> sentinel.
Source dataset label injection	The <code>source_dataset</code> column (<code>'dataset3'</code> or <code>'dataset1'</code>) does not exist in any source file and is injected by the ETL orchestrator, enabling downstream queries to filter by data vintage or combine both sources with explicit awareness of the difference.

5.6 Final Row Counts

After a complete ETL run, the warehouse contains approximately:

Table	Rows	Source
<code>Dim_Date</code>	2 192	Programmatically generated
<code>Dim_Airline</code>	14+	<code>airlines.csv</code> + Dataset 1 supplement
<code>Dim_Airport</code>	322	<code>airports.csv</code>
<code>Dim_Plane</code>	~4 800	Scanned from <code>flights.csv</code>
<code>Dim_Cancellation_Reason</code>	5	DDL seed
<code>Fact_Flight_Operations</code>	~8 800 000	Dataset 3 (~5.8M) + Dataset 1 (~3M)